D2 Detailed Design: Calorie-Based Meal Finder

Assignment 7 | Team members: Eric Braverman, Brayden Molinyawe, Christopher Farah

1. Header, conventions, and D2 overview

Project title: Calorie-Based Meal Finder

Goal statement: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Reviewer overview: A logged-in user enters a calorie target and gets stored complete meals or grocery items that fit it. C1 is the Next.js web frontend; C2 returns calorie-ranked results; C3 manages the saved target and account deletion; C4 imports validated nutrition data; C5 is PostgreSQL. E1 Keycloak owns identity, E2 USDA provides grocery data, and E3 Cloudflare Tunnel is the HTTPS entry point to the single Debian VM. D1 opened C2/C4/C5: nutrition entities, search and import algorithms, API payloads, reuse decisions, and stack choices. D2 finishes C1/C3 and the shared authentication, configuration, logging, dependency, and recovery behavior. It adds a category-complete error table, requirements traceability, design rationale, and review actions.

Conventions: Each diagram repeats the project title and goal. Blue rounded boxes are modules/actions; green boxes are persistent app data; gray dashed boxes are configured external systems; yellow boxes are states/outcomes. Solid arrows show requests, data, or events; red dashed arrows show failure paths. The profile figure labels associations with counts at both ends; it does not claim cross-system database foreign keys. C/E/I IDs retain D0's identifiers. New REQ IDs below identify design-derived requirements when the supplied stories give no AC number; they are not claimed to be original course IDs.

Design closure: Every built D0 component has a data treatment, behavior, and interface in D1 or D2. E1/E2/E3 are configured/integrated, not rebuilt. Their message boundaries and failure handling are documented. The remaining open product decisions are US-06 through US-08 macro ranking/preferences, guest search, and whether the team wants later scale beyond the class-project sizing assumption. They are explicitly outside this release. Exact dependency versions, actual VM performance, and live deployment settings must be verified during implementation, not invented here.

Review status: The supplied materials contain no actual reviewing-team feedback or review-session record. Section 6 records a completed document-level self-review and a separate pending entry for real peer comments. The team must add those comments and actual review outcomes before calling the peer-review portion finished.

2. Finished components, the D2 diagrams

C1 Web Frontend: data, behavior, and interfaces

Data: C1 has no persistent app-owned domain store. Browser state consists of SearchForm {targetText: string, type: MEAL/GROCERY, restaurantIds: UUID[]}, SearchState {phase: READY/SEARCHING/RESULTS/EMPTY/ERROR, requestId: int, response: SearchResponse or null, error: ErrorResponse or null}, and a selected result ID. Import preview data is durable in C5, not browser storage. Profile values come from I6. The opaque session cookie points to a bounded, volatile server-side session map containing E1 tokens and expiry; no tokens are sent to client JavaScript.

Behavior: Require login before search. After login, GET the saved target and prefill only when present. Validate whole-number target text for usability, then submit; C2 repeats all validation. Render results as a semantic list; opening details uses the already-returned item list, so selection cannot silently pull a different catalog version. Use request IDs plus cancellation to prevent an old response replacing a newer search. Show loading, errors, no matches, above-budget alternatives, and import previews as explicit states. Saving is an explicit action, not a side effect of every keystroke. Account deletion requires a clear destructive-action confirmation in the UI.

Accessibility: Each input has a visible programmatic label and useful field-error association. Keyboard users can reach filters, search, details, save, and delete actions; focus indicators remain visible. Announce result count/loading/empty state through a polite live region, and announce errors through an alert. Each result exposes its name, type, calorie count, and serving size as text; details expose every component and quantity. Use the exact label "calorie match" and no "healthy" claim. Do not announce identical result content twice or move focus on every render.

Interfaces: I2 receives HTTPS-origin requests forwarded by E3; I4 performs OIDC; I5/I6/I7 are proxied only to the fixed internal backend, never to an arbitrary caller-supplied URL. C1's same-origin `/api/v1/...` routes preserve the D1/D2 backend JSON contracts and attach server-side tokens. `/login` starts OIDC, `/auth/callback` consumes code/state, `/logout` invalidates the C1 session and starts E1 logout. Login callback is more specific than the `/auth/*` E1 ingress rule and routes to C1. All browser state-changing calls require a valid session, same-origin Origin header, and synchronizer CSRF token in `X-CSRF-Token`. Forward no caller-supplied Authorization header through the browser BFF.

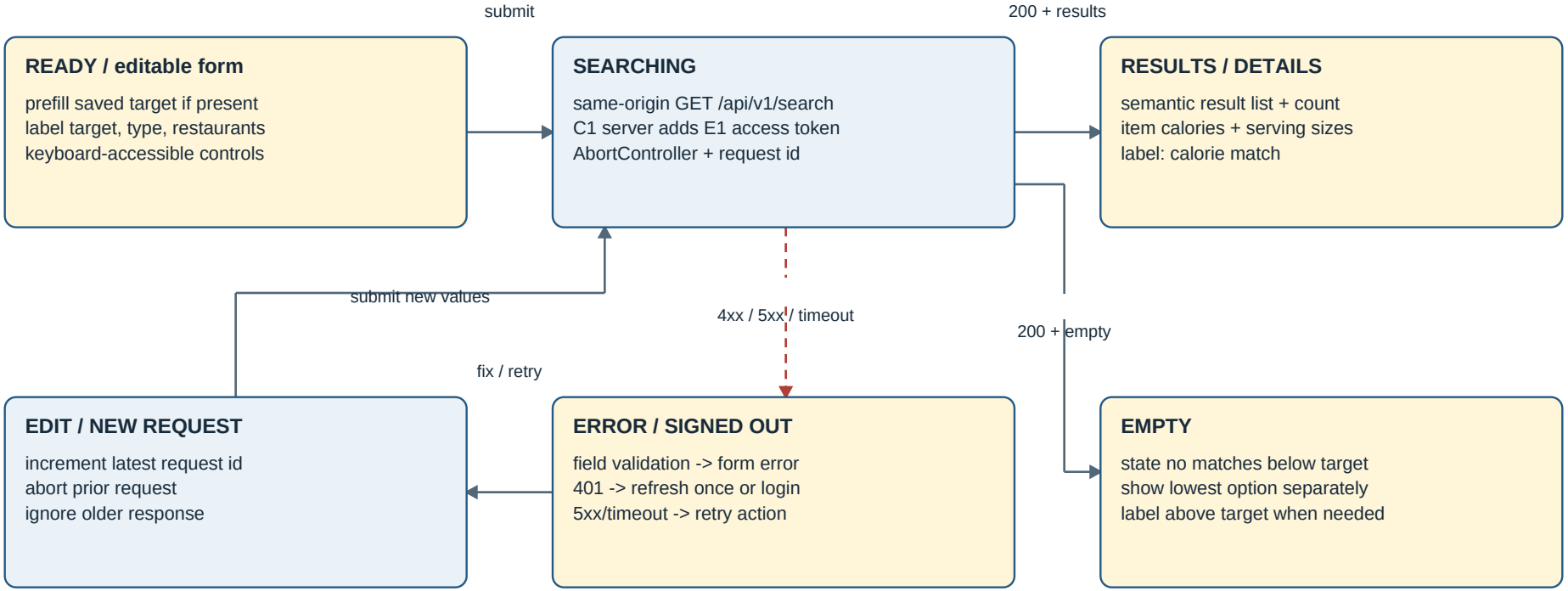
Internal signatures: `submitSearch(SearchForm) -> Promise<SearchState>`; `renderResult(SearchResult) -> semantic HTML`; `saveTarget(int) -> Promise<UserProfile>`; `confirmImport(UUID) -> Promise<ImportResponse>`; `deleteAccount() -> Promise<void>`. Server handlers use `getSession(opaqueCookie) -> Session or null` and return JSON errors or a login redirect as appropriate.

D2 frontend state diagram

Calorie-Based Meal Finder | D2 frontend behavior (C1)

Goal: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Yellow boxes = browser states; blue boxes = server handlers. Arrows show events/data; red dashed arrows show errors.



aria-live polite: loading, result count, empty state. role=alert: errors. Focus stays stable on normal results.

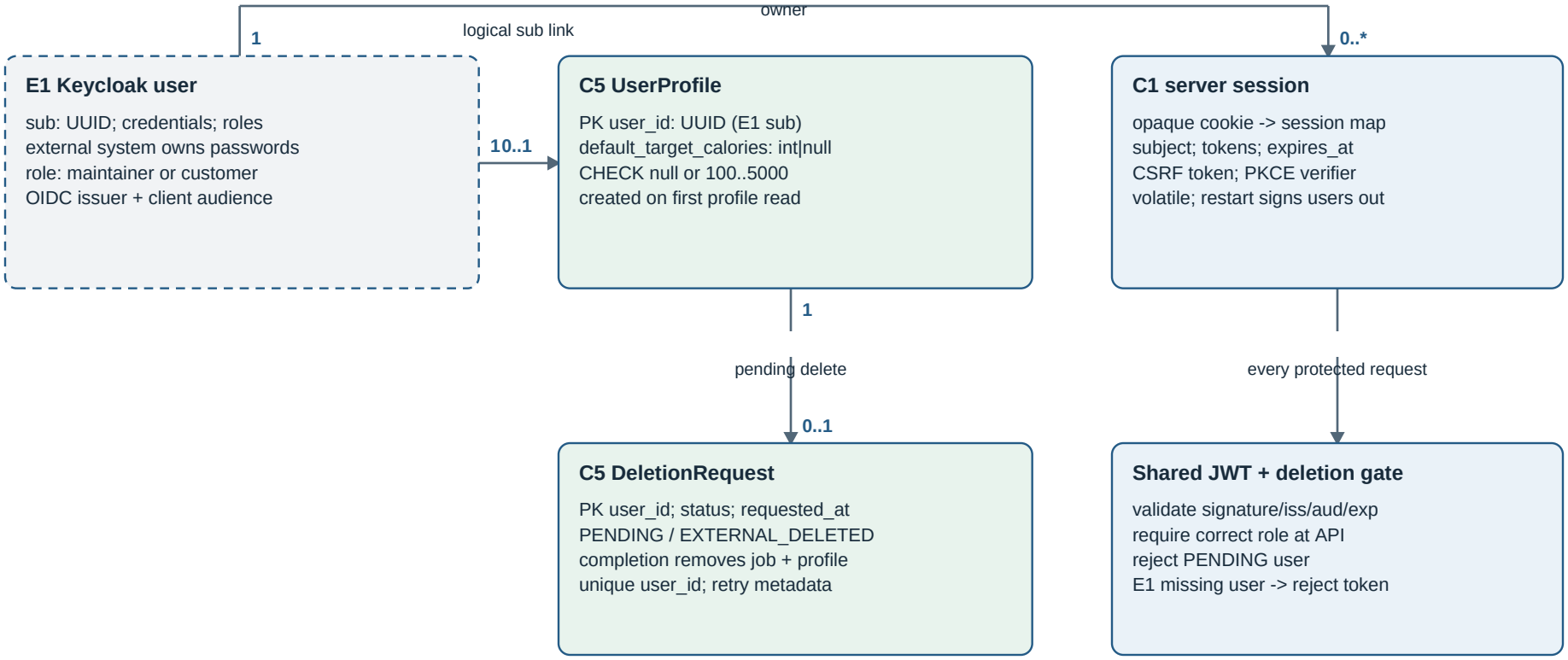
Form validation repeats on C2. Raw OIDC access/refresh tokens stay on the C1 server, outside browser storage.

C3 User Profile API: data model and boundaries

Calorie-Based Meal Finder | D2 profile and auth data (C3/C1)

Goal: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Green = app persistence; gray dashed = external E1 persistence; blue = volatile session. Lines label associations, not DB FKs.



E1 sub is a logical reference, never a cross-system foreign key. C5 stores no password, email, or search history.

DeletionRequest is durable coordination data; success removes it. No macro preferences in the baseline schema.

C3 behavior and I6 contract

Persistence: UserProfile {user_id: UUID PK = E1 sub, default_target_calories: nullable int 100..5000}. No passwords, email, food logs, or search history. The first profile GET inserts an empty profile only for a verified existing E1 user; a race uses `INSERT ... ON CONFLICT DO NOTHING`. DeletionRequest stores the minimum durable coordination information shown in the figure and is removed when deletion finishes. A failed deletion retains the original target until retry succeeds, but app use is blocked while it is pending. The pending state is not presented as successful deletion.

Method / path	Inputs and outputs	Error behavior
GET `/api/v1/profile/target`	JWT subject required; no body/query. 200 `{userId: UUID, defaultTargetCalories: int or null}`. For an existing E1 user with no profile, create and return null target.	401 AUTH_REQUIRED; 409 ACCOUNT_DELETION_PENDING; 503 AUTH_UNAVAILABLE or PROFILE_UNAVAILABLE.
PUT `/api/v1/profile/target`	JSON `{defaultTargetCalories: int}` required, 100..5000 inclusive. User ID is always taken from the JWT, never from JSON. 200 same profile shape after commit.	400 VALIDATION_ERROR; 401 AUTH_REQUIRED; 409 ACCOUNT_DELETION_PENDING; 503 AUTH_UNAVAILABLE or PROFILE_UNAVAILABLE.
DELETE `/api/v1/profile`	No body/query. Authenticated current subject; re-check E1 existence and require session CSRF at C1. 204, no body, only after both identity and local data are removed.	401 AUTH_REQUIRED; 502 IDENTITY_DELETE_FAILED on E1 failure/unknown outcome; 503 PROFILE_UNAVAILABLE on local failure. Pending requests may be retried with a still-valid signed token through the deletion-only exception to the ordinary pending-user gate.

All errors use D1's ErrorResponse. C1 consumes 204 without parsing JSON, invalidates all sessions for that subject in its session map, clears the cookie, and shows a signed-out confirmation. If recovery completes without a connected browser, E1/deletion checks deny the stale sessions; C1 clears them on their next request or expiry. Profile requests/responses are `Cache-Control: no-store`. GET does not overwrite a saved value with null. PUT validates server-side and is a single-row upsert. The subject cannot update another account. Profile writes lock the user coordination row or use an equivalent transaction-scoped user lock, preventing a save racing with deletion from recreating a removed profile. After final deletion, a previously issued JWT fails the E1 existence check before any lazy profile creation.

```
PUT /api/v1/profile/target
Authorization: Bearer <access-token>
Content-Type: application/json

{"defaultTargetCalories":700}

{"userId":"44444444-4444-4444-8444-444444444444",
 "defaultTargetCalories":700}
```

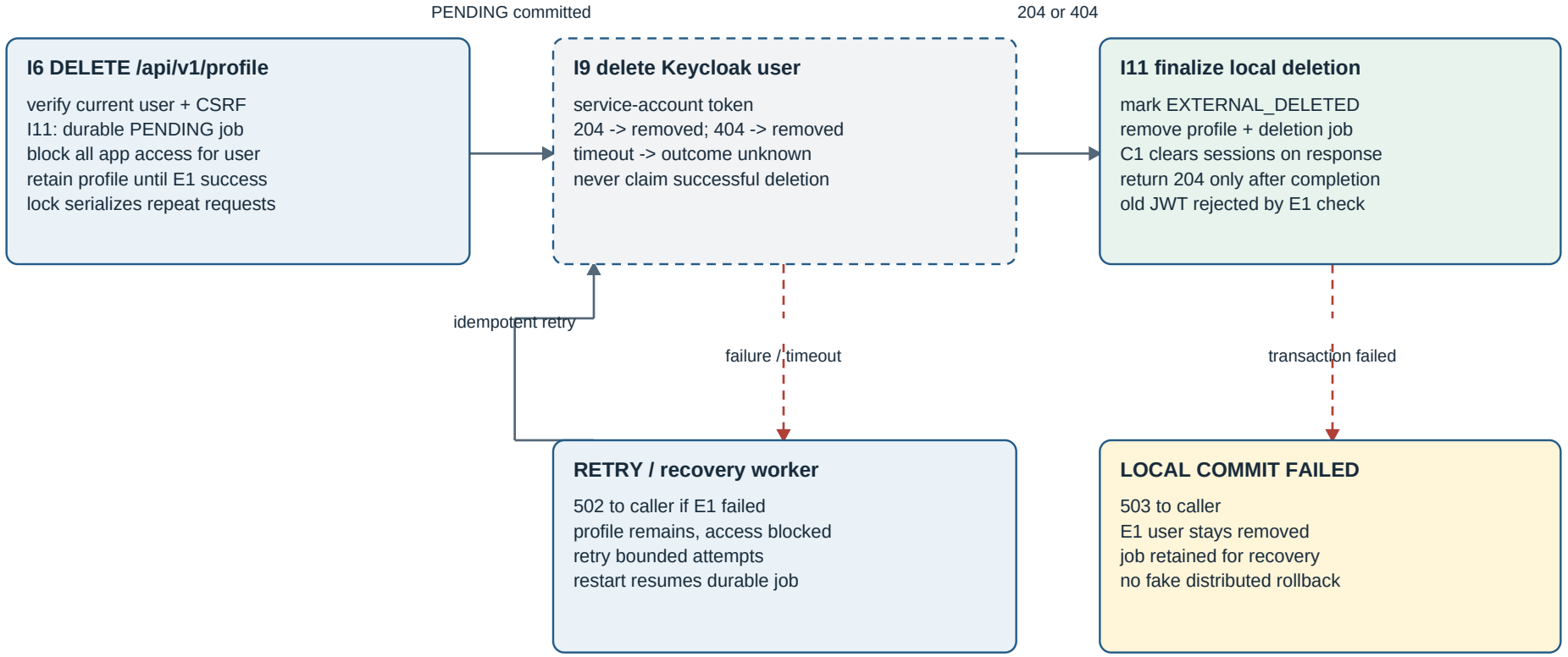
Internal methods: `getOrCreateTarget(subject: UUID) -> UserProfile`; `setTarget(subject: UUID, calories: int) -> UserProfile`; `deleteAccount(subject: UUID) -> void`. Use `ProfileRepository` for I11 and `IdentityAdminClient.deleteUser(subject) -> REMOVED/ALREADY_ABSENT` for I9. `DeletionRecovery.resumePending(limit: int) -> counts` resumes durable jobs on startup and then every minute. Each job gets up to three E1 attempts per run; after ten failed runs it remains blocked and requires an operator's explicit retry, with an alert. No silently abandoned deletion is called complete.

Deletion coordination and revised D0 behavior

Calorie-Based Meal Finder | D2 deletion recovery (C3 + E1)

Goal: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Blue = actions/states. Red dashed arrows = failures. I6/I9/I11 are D0 interfaces; the durable job closes the crash gap.



E1-first deletion plus durable local recovery replaces D0's impossible assumption of a cross-system rollback.

Shared authentication and authorization (E1, C1, backend)

C1 uses Authorization Code with PKCE, state, and nonce through an established OIDC client. Keep the code verifier and expected callback state in a short-lived server session; consume them once. Use an opaque random session ID with a Secure, HttpOnly, SameSite=Lax cookie, Path=/, and no Domain attribute; rotate it after login. Expire an idle session after 30 minutes and at eight hours absolute duration. Keep access/refresh tokens only server-side. An unshared in-memory session map is sufficient for the one C1 instance; cap it at 1,000 sessions, expire old entries, and return 503 rather than evicting an active user's session unpredictably. Restart signs users out. Multiple frontend replicas would require a shared session store or explicit affinity and are outside this deployment.

Refresh an expired access token once using a per-session single-flight lock so parallel searches do not race refresh-token rotation. If refresh fails, clear the session and require login; do not loop. Spring Security verifies JWT signature, allowed algorithm, issuer, audience, expiry and subject, then maps the maintainer role. Missing/invalid/expired credentials give 401; a valid non-maintainer gets 403 on imports. Never infer role from a browser field. The proposed E1 access-token lifetime remains D0's five minutes.

I8 retrieves `/realms/meal-finder/protocol/openid-connect/certs` as JWKS. Cache keys for up to ten minutes as in D0, permit one immediate refresh for an unknown kid, and reject an unresolvable key. A previously trusted cached key can verify an existing kid during an E1 JWKS outage while within cache policy; missing usable keys give 503. This does not bypass the account-deletion/existence gate.

Deletion safety gate: On each protected API operation, read pending deletion state and ask E1's private admin API whether the subject still exists; do not positively cache this result in the baseline. This adds an internal E1 call but closes the five-minute stale-JWT window after deletion, including after the local job has been removed. E1 404 -> 401; timeout/error -> 503; pending local deletion -> 409 except the authorized delete-retry path. A database lookup failure also fails closed. A gate check and local profile write share the user lock for coordination with app-initiated deletion. The pre-query authentication/identity work must fit within D0's validation-stage budget; benchmark it and revise the design if it cannot. Configuring immediate revocation through another mechanism is a later alternative, not an unimplemented claim.

I9 uses client credentials for a narrowly scoped service account with only required realm user read/delete capability. Request a short-lived admin token; keep it server-only, retry acquisition once on expiry, and never log it. The user deletion response cannot roll back a remote E1 deletion. First ensure an empty local profile exists if needed and commit a durable PENDING job; then delete E1 and atomically remove the profile/job. In that local cleanup transaction, clear the subject from any retained import attribution (preserve nonpersonal nutrition provenance). C1 owns session invalidation after a response; disconnected sessions become unusable immediately through the gate and expire from memory. An E1 failure leaves the target unchanged and the job pending. A local failure after E1 success leaves the durable job for recovery. This explicitly revises D0's simplified rollback wording.

E2 nutrition adapter and E3 routing

E2 data and interface: No app-owned database lives inside the USDA client. Its transient records are FDC ID, description, nutrient records, serving quantity/unit, and any documented mass/basis needed for conversion. I13 calls the official HTTPS food-search endpoint for candidates and food-details endpoint for selected IDs. Food search accepts query, pageNumber, pageSize (100), optional dataType list, and a server-only API key. Details accepts selected FDC IDs. Map upstream JSON to typed records; missing, malformed, unsupported, or ambiguous serving-energy data becomes a row error. FoodData Central's documented nutrient basis must be checked per data type; do not treat every energy number as per labeled serving. I13 is expanded from D0's search-only sketch to search plus details for this reason.

E2 behavior: Set a 10-second connect/read deadline per attempt. Retry transport failures, 429, and 5xx at most three times after the initial request, with exponential delays of 1, 2, and 4 seconds plus bounded jitter; honor a shorter or longer Retry-After up to a 30-second foreground operation budget. If waiting would exceed that budget, return 429 SOURCE_RATE_LIMITED with a retry suggestion instead of sleeping for an hour. Persist failure metadata on the batch where created, but never change the live catalog. Capture all pages the maintainer selected; a partial upstream retrieval is not silently treated as a complete selection. D0's current USDA limit is corrected to the provider's documented default per IP, not per API key. User searches use stored rows and continue if USDA is down.

E3 interfaces and behavior: I1 is browser HTTPS to the Cloudflare edge. I2 forwards the app hostname to C1:3000 on the private Docker network. I3 forwards Keycloak `/auth/*` requests to E1:8080, except the exact C1 callback route. Configure the public Keycloak issuer/base URL to match the externally visible `/auth` path; do not validate against an internal hostname in one component and a public hostname in another. Only C1 exposes user-facing application/API handlers through the tunnel; C2/C3/C4 and PostgreSQL have no public host bindings. Responses return on the original connection. Cloudflare terminates TLS; the outbound tunnel is encrypted; local container HTTP is within the private host network as D0 specifies.

Data: E3 has no app-owned persistent user data. Store only its non-secret ingress configuration in the repository; its tunnel credential is a deployment secret. E1 persists its own identity data in a separate logical database/role and volume; the app does not read its tables directly. Keycloak credentials/roles are managed through supported OIDC/admin interfaces. Proxy headers are accepted only from the configured ingress path; never authorize a user from CF-Connecting-IP or X-Forwarded-* headers.

Notifications: There is no email/SMS/push requirement. Notifications in this release are in-page accessible status/error messages and operator log/health signals. No notification service, message broker, or hardware component is added.

Configuration, logging, and operations (C1-C5, E1-E3)

Configuration: Validate required settings at startup: public app URL, public OIDC issuer, audience/client IDs, internal E1/backend URLs, database URLs/roles, allowed origin, USDA key, tunnel credential path, and timeout limits. Supply secrets through protected runtime secret files/environment injection; commit only `.env.example` with key names and no values. Fail startup for missing secrets or inconsistent issuer/callback settings. Pin container/dependency versions; separate app schema migrations from Keycloak's internal database lifecycle.

Logging: Generate a request UUID at the first app handler and propagate it internally. Log timestamp, request ID, component, route template, duration, status, and stable error code. An import may log batch ID, source type, row-error count, and summary counts. Deletion logs record job lifecycle without persistent user identifiers after completion. Never log cookies, Authorization headers, password/code/token values, API keys, complete uploaded records, query strings, or saved calorie targets. Keep ordinary app logs for seven days with size rotation; restrict access. PostgreSQL and proxy logging must follow equivalent redaction. Import/deletion errors shown to users carry the request ID for support.

Health and restart: Expose `/internal/health/live` for process liveness and `/internal/health/ready` for configured DB/schema readiness only on the private network. Do not expose admin/health internals through the public BFF. DB unavailable -> backend unready and ordinary APIs return 503. E1 unavailable -> protected requests fail closed; USDA unavailable -> imports fail but stored search remains available if E1/DB work. Configure Compose restart policies and health-gated startup; handle runtime failures as well as startup ordering. Recover PENDING deletion jobs and nonterminal import states on restart; a crashed confirm reloads the persisted status before deciding whether to retry.

Persistence and backup: Use persistent volumes for C5 and E1. Make a daily encrypted backup of both logical databases and required non-secret configuration, retain seven daily copies, and test restoration before the release demonstration. Keep backups outside the VM. A backup is a proposed operational target, not an existing service or demonstrated recovery guarantee. Do not store raw search history. Completed deletion removes the live profile immediately; encrypted backups age out within seven days, and any restoration must reapply deletions occurring after the restored snapshot before app access is reopened. Retain minimal protected deletion tombstones outside routine restored app data for that seven-day recovery window, then erase them. Access to this recovery ledger is administrative, and it must not be used to recreate an account.

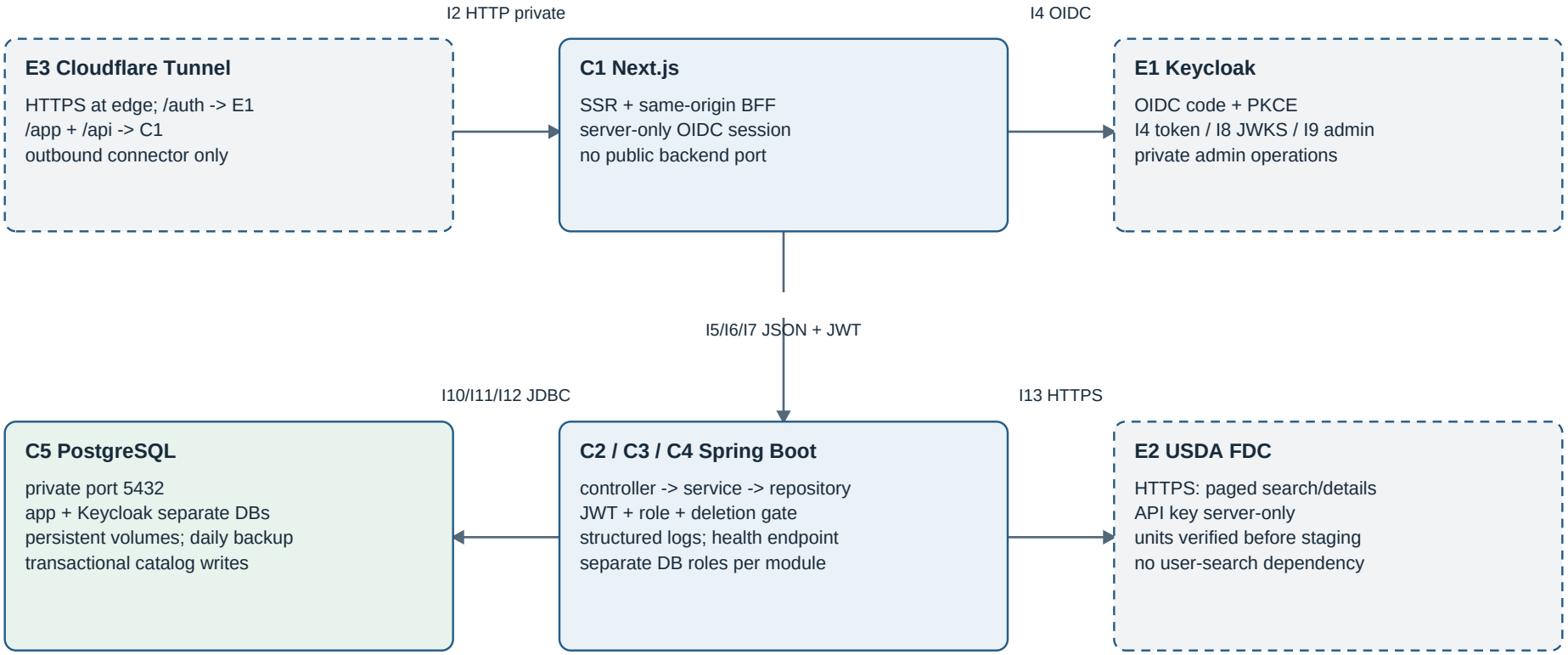
I10-I12 update: I10 becomes an explicitly repeatable-read snapshot operation. I11 adds deletion jobs and user-lock coordination alongside the target row. I12 includes persistent import staging, catalog-version locking, and atomic total/status updates. These internal changes preserve D0 interface IDs and are recorded in Section 5. C2 has SELECT-only access to catalog and deletion-gate data; C3 writes profile/deletion tables and may execute a narrowly scoped function to null its deleted subject's import attribution; it cannot change nutrition values. C4 writes catalog/import tables. Admin migration credentials are not used by runtime requests.

D2 integration diagram

Calorie-Based Meal Finder | D2 deployment and integration

Goal: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Blue = app build; green = persistence; gray dashed = configured external system. Arrows label protocols/interfaces.



Dashed VM boundary omitted for space: C1, backend, C5, E1 and cloudflared run on the same Debian VM.

Browser -> E3 is I1; E3 -> E1 is I3; backend -> E1 is I8/I9. Responses return along each request path.

3. Edge cases and error handling

Each D0 built component covers all four required categories: **boundary values, timing/concurrency, missing input, and external dependencies**. Additional rows cover E1-E3. IDs are design references. Handler names denote the component that owns the response, even when the UI also renders it. D2_Deletion_State and D1_Import_State show error paths graphically.

ID / component	Category and concrete case	Decision / response	Handler / reversibility
EC-01 C1	Boundary: target 99, 5001, -50, 100.5; result count 0.	Show linked field error; accept 100 and 5000. Empty results have a visible/announced state.	C1 form; reversible by edit; C2 revalidates.
EC-02 C1	Timing: earlier search finishes after a later search; double confirm.	Abort old request and check sequence ID; disable pending confirm and rely on server idempotency.	C1 request state; old response discarded, no catalog change.
EC-03 C1	Missing: blank target, null saved target, no session cookie.	Require target; null does not become zero. Missing session -> login.	C1; no API search sent; no app data changed.
EC-04 C1	Dependency: HTML gateway error or backend response exceeds client deadline.	At 2 seconds leave loading and surface a retry state; cancel if possible. Do not claim the requirement was met.	C1; GET can retry; a timed-out mutation first reads server state.
EC-05 C2	Boundary: exact match, zero-calorie items, equal-calorie ties.	Include <= target, accept zero, sort by calories descending then ID ascending.	C2; read-only.
EC-06 C2	Timing: import commits while meal components are loading.	One REPEATABLE READ snapshot for IDs, components, and fallback. SQL 500 ms limit.	C2 repository; rollback read transaction on timeout.
EC-07 C2	Missing: type/target absent, malformed UUID, repeated singleton input.	400 VALIDATION_ERROR; no catalog query. GROCERY with restaurant filter also rejected.	C2 controller; no mutation.
EC-08 C2	Dependency: database down, unknown signing key, E1 gate unavailable.	503 CATALOG_UNAVAILABLE or AUTH_UNAVAILABLE; never return an unverified or partial list.	C2 + shared security; retry is user initiated.
EC-09 C3	Boundary: target outside 100..5000; GET with no local profile.	PUT 400; first verified GET creates a null target; no caller-specified user ID accepted.	C3; failed write rolled back.
EC-10 C3	Timing: save races delete; crash after E1 deletion.	User lock prevents resurrection; durable job resumes local cleanup.	C3; remote deletion is irreversible, local cleanup is retryable.
EC-11 C3	Missing: empty PUT body/target; missing E1 user.	400 for malformed target; E1 404 at authentication -> 401; no lazy profile creation.	C3/security; no profile recreation.
EC-12 C3	Dependency: E1 delete fails or DB finalization fails.	502 leaves pending job/profile; 503 after E1 success retains recovery job. 204 only on full completion.	C3 recovery worker; never claim a remote rollback.

Error table, continued

ID / component	Category and concrete case	Decision / response	Handler / reversibility
EC-13 C4	Boundary: >5 MiB, >20,000 rows, duplicate key, empty snapshot, negative kcal.	413 for byte cap; 422 for row/semantic errors; reject whole input. Apply row/string limits before unbounded allocation.	C4; live data unchanged.
EC-14 C4	Timing: two previews confirm against same catalog version; duplicate confirmation.	CatalogState lock; first wins, stale second 409; already APPLIED returns saved 200.	C4; one mutation per batch.
EC-15 C4	Missing: serving size, calories, source key, USDA serving mass/basis.	422 with exact row/index and reason; no default zero or guessed units.	C4 normalizer/validator; nothing applied.
EC-16 C4	Dependency: USDA 429/5xx or DB failure during commit.	Bound retries; 429/502 on source failure; rollback all catalog changes on SQL failure, then persist FAILED if possible.	C4; if commit outcome unknown, reload batch state before marking failed or retrying.
EC-17 C5	Boundary: quantity <= 0, orphan component, duplicate scoped key, sum overflow.	DB constraints plus service-level aggregate range check reject transaction; counts/calories must fit signed int.	C5 constraints/C4; whole write rollback.
EC-18 C5	Timing: deadlock, serialization failure, or lock timeout.	Retry an idempotent confirmation transaction once after re-reading status/version; otherwise 503 and no partial commit.	Repository/service; duplicate status protects completed work.
EC-19 C5	Missing: uninitialized schema, missing CatalogState, orphan meal definition.	Fail readiness/migration; active meal creation must include components in same transaction.	Startup migrator + C5; do not serve an invented empty catalog.
EC-20 C5	Dependency: disk full, connection loss, host restart.	503; persistent volumes, rollback/recovery, readiness fails. Restore from backup only with deletion replay.	C5/operator; uncertain commits reconciled by persisted batch/job IDs.
EC-21 E1/security	Boundary: expired JWT, wrong issuer/audience/algorithm, unknown kid.	401 invalid credential; one key refresh for kid; 503 if verification dependency cannot be obtained.	Shared security; no protected action.
EC-22 E1/security	Timing: parallel refresh, replayed callback, deletion with old JWT.	Single-flight refresh; state/nonce/verifier consumed once; existence gate rejects removed identities.	C1 + security; no positive existence cache.
EC-23 E1/security	Missing: maintainer role, subject, client secret.	403 for valid user missing role; 401 for invalid subject; absent required secret fails startup.	Security/configuration; no import/deletion privilege inferred.
EC-24 E1/security	Dependency: E1 unreachable or admin credential rejected.	Fail closed with 503 for ordinary authentication; delete operation 502; alert operator on service-account misconfiguration.	Identity adapter; no endless token retry.

Error table and validation scenarios, continued

ID / component	Category and concrete case	Decision / response	Handler / reversibility
EC-25 E2	Boundary: 1,001 selected IDs, calorie unit not convertible, oversized response.	Reject selection; enforce adapter response cap of 10 MiB and typed unit checks; 422 row error or 502 malformed upstream.	C4 USDA adapter; no staging accepted as valid.
EC-26 E2	Timing: Retry-After exceeds 30-second import budget; page retrieval interrupted.	Stop bounded foreground operation; return 429/502 with retry instruction; do not use partial selection.	C4; retry creates/reuses an uncommitted preview, never auto-applies.
EC-27 E2	Missing: no energy nutrient, missing serving size, absent API key.	Row-specific 422 for missing nutrition; missing configured key fails import readiness/configuration.	C4/configuration; no kcal guessed.
EC-28 E2	Dependency: 429, 5xx, DNS/read failure.	Initial attempt plus <= 3 retries within budget; ordinary 4xx is not retried. Source failure cannot delete catalog rows.	C4 HTTP adapter; stored search unaffected.
EC-29 E3	Boundary: wrong host/path or excessive request body.	Default ingress rejects unknown host/path; C1 also enforces upload cap; unsupported content gives 415.	E3/C1; no arbitrary internal routing.
EC-30 E3	Timing: tunnel reconnect during import confirmation.	Connector restarts/reconnects. After reconnection GET batch status before resubmitting a mutation.	E3 restart policy + C1; server idempotency prevents duplicate apply.
EC-31 E3	Missing: connector credential or ingress mapping.	Fail connector startup; deployment check verifies app and exact callback/auth routes.	Deployment owner; backend remains private.
EC-32 E3	Dependency: edge/tunnel or C1 unavailable.	Provider gateway page/502/1033 may occur; alert via health/deployment monitoring; no automatic rollback of a committed operation.	E3/operator; recover service, then reconcile mutation status.

Design validation scenarios, to execute during implementation: Seed 450/680/900 kcal meals and verify target 700 -> 680 then 450, target 300 -> empty plus 450 fallback, 100 and 5000 accepted, and malformed targets cause no catalog query. Change an item from 500 to 550 used once in each of two meals and verify both totals increase by exactly 50. Verify a preview with one add, one remove, two changes has exactly those counts. Reject CSV lines 7 and 12 for the stated bad rows without changing any foods/totals. Race two confirmations and retry a completed one. Simulate a crash between E1 deletion and local cleanup and verify recovery and old-token rejection. Check result announcements with NVDA or VoiceOver and keyboard-only use. These scenarios are traceable design checks, not claims of executed application tests.

4. Requirements traceability table

Rows map to exact requirements visible in the supplied baseline. G-01 through G-04 are local labels for rubric/architecture constraints described in D0; REQ-* labels identify measurable derived requirements where no original AC number was supplied. Macro-extension criteria AC-06/07 are not silently added to the calorie baseline.

ID / requirement	Design element that satisfies it	Component / interface	Verification planned
US-01, AC-01.1: 700 target returns 680 then 450 within 2 s	A1 cutoff, descending calories, deterministic tie rule; indexed snapshot read and D0 timing allocation.	C1/C2/C5; I5/I10; D1_Search_Flow.	Seed 450/680/900; inspect result order and measure submit-to-render latency on deployed VM.
AC-01.2: details show each item, calories, serving; total agrees; calorie-match label	SearchResult items, quantity, explicit rounding adjustment; details use returned snapshot.	C2 aggregation/model; C1 details renderer; I5.	Sum displayed component contributions and adjustment; inspect exact label, absence of healthy claim.
AC-01.3: no matching meals shows message and lowest meal	Empty results plus scoped closestOption; separate above-target alternative UI.	C2 fallback query, C1 EMPTY; I5/I10.	Target 300 on fixture returns 0 matches and 450 fallback; empty restaurant returns null.
AC-01.4: invalid target runs no search; accepted range stated	Repeated client/server validation before repository access; error envelope.	C1/C2; I5, EC-01/07.	-50 and abc rejected; repository not called; message says whole number 100..5000.
AC-01.5 / UC-01 A1: save and prefill 700 after login	Subject-keyed UserProfile, explicit PUT, GET on next login.	C1/C3/C5; I6/I11.	Save 700, logout/login; confirm prefilled 700 and isolation from another user.
US-02 / REQ-GROCERY-01: compare calories per stated grocery serving	FoodItem resultType GROCERY; normalized per-serving kcal and exact portion text.	C2/C4/C5; I5/I13.	Grocery search returns single-serving item entries; known 100 g example normalizes correctly; ambiguous basis rejects.
AC-03.1: item 500 -> 550 changes each of 2 meals by +50	Affected-meal index, quantity-aware totals, one confirmation transaction.	C4/C5; I7/I12; D1_Import_State.	Item used with quantity 1 in 2 meals; verify both totals +50 and source/time recorded.
AC-03.2: preview exactly 1 add, 1 remove, 2 changes before save	Stable-key diff, full CSV scope, staged PENDING_CONFIRMATION response.	C4/C5/C1; I7/I12.	Inspect counts and unchanged live catalog before confirmation.
AC-03.3: invalid lines 7 and 12 reject entire file	Physical line-number errors, all-row validation before live writes.	C4 validator; I7, EC-15.	Missing serving line 7, negative kcal line 12 -> 422 with both lines; no food or total change.

Traceability, continued

ID / requirement	Design element that satisfies it	Component / interface	Verification planned
UC-02 A1: unchanged file says 0 changed and saves nothing	NO_CHANGES terminal response; no catalog version increment.	C4; I7; A2.	Import identical scope; zero counts and identical DB version/content.
US-04 / REQ-ACC-01: screen reader announces name, type, calories, serving	Semantic list; labels; live result count/status; text details; focus control.	C1 result/details components; D2_Frontend_State.	NVDA/VoiceOver and keyboard review of success, empty, validation, and import status.
US-05 / REQ-PRIV-01: delete account and saved goal; old account cannot log in	Durable delete job, E1-first deletion, local cleanup, session invalidation, no lazy recreation.	C3/C1/E1/C5; I6/I9/I11; D2_Deletion_State.	Verify E1 account/profile absent, prior credentials rejected, issued old JWT blocked.
G-01 / security: only needed personal data; credentials stay in E1	UserProfile stores subject/target only; tokens server-only; redacted seven-day logs; recovery-only deletion ledger expires.	C1/C3/C5/E1; D2 profile/security/operations.	Inspect schema and redacted logs; no token in client storage; verify cleanup/retention settings.
G-02 / economic: one low-cost VM; free public data; no required paid component	D0 stack retained; offline CSV/USDA import; no broker or Kubernetes.	C1-C5/E1-E3; D1 Section 6.	Review actual VM/domain/backup cost and current plan eligibility before deployment.
G-03 / deployment: no public inbound VM ports	Outbound connector; ingress to C1/E1 only; private DB/backend.	E3; I1-I3; D2_Deployment_Glue.	Review Compose port bindings/firewall and publicly reachable routes.
G-04 / authentication: logged-in search and maintainer-only updates	OIDC session, JWT claims + existence/deletion gate; role-based import check.	C1/E1/shared security/C4; I4/I5/I7/I8.	Unauthenticated search 401; customer import 403; maintainer preview allowed.
REQ-ATOMIC-01 / derived: no partial import or stale confirmation	CatalogState lock/version, batch status, full rollback, duplicate confirm result.	C4/C5; I12; EC-14/16/18.	Fault injection, concurrent confirm, expired preview, lost response/retry.
REQ-OPS-01 / derived: dependency failures give defined errors and recovery	EC-01..32, readiness, timeouts, retry budgets, backup/deletion replay.	C1-C5/E1-E3; Sections 2-3.	Dependency outage rehearsal, restart, restore, and status reconciliation.
US-06 / US-07 / US-08: proposed macro extension	Deferred; calories-only path preserved. No protein ranking, macro mutation, or preference endpoint promised by v1.	Future scope only; source macro notes.	Define acceptance criteria and obtain team decisions before implementation; then extend traceability.

Coverage check: US-01..05 each map to a concrete implementation element; each built component appears above. All baseline supplied numbered ACs are mapped, including AC-01.5 visible across the source-image boundary. Every operational requirement is mapped to behavior or configuration instead of only a box name. D0's source does not provide numbered ACs for grocery, accessibility, or deletion, so their derived IDs are explicitly identified.

5. Design rationale

These ten significant decisions link back to D0's log instead of repeating it. DR-06 through DR-09 record explicit D0 interface refinements; they are design changes, not claims of team approval.

ID / decision	We chose X over Y because...	Trade-off, evidence, and D0 reference
DR-01 Relational model	We chose PostgreSQL relations over a document-only store because meal composition, uniqueness, and all-or-nothing imports need explicit integrity rules.	Joins and migrations add schema work; transactions and FK/index rules make correctness reviewable. Extends DL-04; D1 data model.
DR-02 Indexed stored-meal search	We chose filtering curated stored meals over generating combinations because UC-01 asks for complete meals with fixed serving details.	Limited to maintained combos; avoids unbounded search and supports the 2-second budget. Extends DL-02/DL-04; A1.
DR-03 Precomputed totals	We chose import-time totals over summing every meal on every search because reads dominate and calorie order must be indexed.	Import complexity increases; same-transaction aggregation and snapshot reads prevent stale totals. Extends DL-04; A2/I12.
DR-04 Stable-key staged imports	We chose preview/confirm with stable IDs over immediate or name-matched writes because names change and maintainers must see an exact diff.	Requires staging, expiry, and version checks; gives safe refresh/retry and implements AC-03.2. Extends DL-06; I7.
DR-05 Reuse stack and OIDC	We chose D0's Spring modular monolith, Next.js, and Keycloak over new services and custom authentication because the team already knows the stack.	One host/process family is a failure domain; simpler deployment and standard protocol code fit project scale. Retains DL-01/DL-02/DL-03/DL-05/DL-07.
DR-06 Server sessions and accessible states	We chose an opaque cookie plus server-held tokens over browser token storage, and explicit states over an unlabeled changing list.	C1 restart signs users out; state management needs care. Preserves D0 token boundary and US-04. I2/I4 refined with exact callback/CSRF rules.
DR-07 Durable deletion recovery	We chose a durable job with E1-first deletion over an assumed distributed rollback because a committed E1 deletion cannot be rolled back by PostgreSQL.	Pending deletion can temporarily block app access and needs recovery. Revises D0 I6/I9/I11 error wording; D2_Deletion_State and EC-12.
DR-08 Immediate deleted-user gate	We chose a per-request E1 existence check over trusting a valid five-minute JWT alone because US-05 requires removed accounts to lose app access.	Adds E1 latency/availability dependency; fail closed and benchmark it. Refines I8/I9 security integration; alternative revocation design needs separate validation.
DR-09 Source normalization and detail fetch	We chose USDA details + verified serving basis over copying search energy numbers because source kcal may describe 100 g rather than the label serving.	More import requests; known units prevent incorrect results. Expands I13, corrects rate limit to per IP; extends DL-06 and AC-03.3-style validation.
DR-10 Snapshot and version contracts	We chose repeatable search snapshots and version-locked confirmation over independent reads and silent last-writer-wins.	Extra transaction/locking work; predictable results and no stale preview apply. Refines I5/I7/I10/I12 and quantity/rounding response fields; extends DL-04.

6. Peer design review notes

Still needs to be reviewed